

CS-6314: Information Architecture — Complete Study Notes

A detailed, exam-oriented study guide covering all 5 units, with curated reference links (GeeksforGeeks and other trustable sources) for deeper reading on every topic.



Table of Contents

- [Course Outcomes \(COs\)](#)
- [Unit I: Introduction to UX/UI](#)
 - [1.1 What is UX and UI?](#)
 - [1.2 Design Issues in UX/UI](#)
 - [1.3 Tools and Processes of UX/UI Design](#)
 - [1.4 Design Principles of Good UX/UI Design](#)
- [Unit II: Introduction to Information Architecture](#)
 - [2.1 What is Information Architecture?](#)
 - [2.2 Principles of Information Architecture](#)
 - [2.3 Role of the Information Architect](#)
 - [2.4 Areas of Information Architecture](#)
 - [2.5 Types of Architecture](#)
 - [2.6 Research and Practice in IA](#)
- [Unit III: Organizing Information](#)
 - [3.1 Organizing Information & Organizational Challenges](#)
 - [3.2 Organizing Websites and Intranets](#)
 - [3.3 Creating Cohesive Organization Systems](#)
 - [3.4 Organizing the WWW & Browser Navigation Features](#)
 - [3.5 Building Context & Improving Flexibility](#)
 - [3.6 Types of Navigation Systems](#)
 - [3.7 Integrated & Remote Navigation Elements](#)
 - [3.8 Designing Elegant Navigation Systems](#)
- [Unit IV: Labeling Systems & Search](#)
 - [4.1 Labeling Systems — "Not Labels"](#)
 - [4.2 Types of Labeling Systems](#)
 - [4.3 Creating Effective Labeling Systems](#)
 - [4.4 Fine-Tuning the Labeling System](#)
 - [4.5 Non-Representational Labels & The Double Challenge](#)
 - [4.6 Searching a Website — How Users Search](#)
 - [4.7 Designing the Search Interface](#)
 - [4.8 The Reference Interview & Indexing](#)
 - [4.9 To Search or Not to Search](#)
- [Unit V: The Open Group Architecture Framework \(TOGAF\)](#)
 - [5.1 Introduction to TOGAF 9](#)
 - [5.2 TOGAF 9 Management Overview](#)
 - [5.3 TOGAF 9 Components](#)

- [5.4 Introduction to the ADM](#)
- [5.5 ADM Phases](#)
- [5.6 ADM Deliverables](#)
- [5.7 Enterprise Continuum & Architecture Repository](#)
- [5.8 Architecture Governance](#)
- [5.9 Views, Viewpoints & Architecture Building Blocks](#)
- [Quick Revision Sheet](#)

Course Outcomes (COs)

CO	Description
CO1	Develop conceptual understanding of UX/UI design and information architecture.
CO2	Build a theoretical foundation on principles and concepts of designing effective information architecture.
CO3	Study existing portals on organizing, classification, navigation, searching & labeling; prepare comprehensive strength/weakness reports.
CO4	Understand TOGAF for building information architecture documents.
CO5	Supplement fundamentals with case studies of modern web infrastructure — Cloud, Security, and Mobile.

Unit I: Introduction to UX/UI

1.1 What is UX and UI?

User Interface (UI) is everything a user visually interacts with on a screen — buttons, icons, typography, spacing, color, and layout. It is concerned with **how a product looks**.

User Experience (UX) is the overall feeling and journey a person has while using a product — how easy it is, how satisfying it is, and whether it solves their problem. It is concerned with **how a product works and feels**.

A simple analogy: if a website is a house, UI is the paint, furniture, and interior decoration, while UX is the layout of rooms, how easily you move between them, and whether the house actually meets your needs.

Key distinction to remember for exams:

- UI = visual/interactive layer (look and feel)
- UX = the entire journey (research → wireframe → prototype → testing)
- UX is broader and includes UI as one of its outputs.

Study more:

- [Why to use UI/UX Design? — GeeksforGeeks](#)
- [UX Design | Key Process, Flow and Principles — GeeksforGeeks](#)

1.2 Design Issues in UX/UI

Common issues that a designer must anticipate and resolve while designing UX/UI:

1. **Inconsistent design language** — different fonts, colors, and button styles across the same product confuse users.
2. **Poor information hierarchy** — when everything is given equal visual weight, users cannot tell what is important.
3. **Ignoring accessibility** — not designing for users with visual, auditory, motor, or cognitive disabilities excludes a large user base.
4. **Cognitive overload** — cramming too many options, choices, or pieces of information onto one screen.
5. **Lack of responsiveness** — designs that break or become unusable across different screen sizes (mobile, tablet, desktop).
6. **Ignoring user feedback loops** — not giving the user confirmation after an action (e.g., no confirmation after form submission).
7. **Designing for the designer, not the user** — relying on personal taste rather than user research and testing.
8. **Slow performance** — even a beautiful UI fails if load times are high, hurting the overall experience.

These issues are usually addressed through **usability testing**, **heuristic evaluation**, and following established design principles (see 1.4).

Study more:

- [Fundamentals of Solid UI/UX Design — GeeksforGeeks](#)
 - [Designing User-Friendly Interfaces: Essential UX Principles — GeeksforGeeks](#)
-

1.3 Tools and Processes of UX/UI Design

The UX/UI Design Process typically follows these stages:

1. **Research** — understanding users through interviews, surveys, and competitor analysis.
2. **Define/Analyze** — turning research into personas, user journeys, and problem statements.
3. **Ideate** — brainstorming solutions, sketching, creating information architecture and site maps.
4. **Design (Wireframe → Prototype → Visual Design)** — moving from low-fidelity wireframes to high-fidelity, interactive prototypes.
5. **Test** — usability testing, A/B testing, and gathering feedback.
6. **Implement & Iterate** — developers build the product; designers keep refining based on real usage data.

Common Tools used at each stage:

- **Research & Wireframing:** Figma, Adobe XD, Sketch, Balsamiq
- **Prototyping:** Figma, InVision, Adobe XD, Framer
- **User Testing:** Maze, UsabilityHub, Hotjar
- **Collaboration/Handoff:** Zeplin, Figma Dev Mode, Miro (for journey mapping)

This maps directly to **Design Thinking**, a five-step methodology: **Empathize** → **Define** → **Ideate** → **Prototype** → **Test**.

Study more:

- [How to use Design Thinking principles in UI/UX Design? — GeeksforGeeks](#)
- [UX Design | Key Process, Flow and Principles — GeeksforGeeks](#)

1.4 Design Principles of Good UX/UI Design

The core principles you should be able to explain with examples in an exam:

Principle	Meaning
Simplicity	The design should require minimal effort to use and understand.
User-Centered Design	Designed around the actual needs of the user, including users with disabilities.
Visibility	Important tasks and options are clearly visible, not hidden.
Consistency	Same colors, typography, and layout patterns used throughout — builds trust and reduces learning curve.
Feedback	The system tells the user the result of their action (e.g., a success message after form submission).
Clarity	Users should immediately understand what a screen or element does.
Accessibility	Usable by everyone, including people using screen readers or keyboard-only navigation.
Efficiency	The design should let users complete tasks quickly, with minimal steps.
Hierarchy	Visual weight (size, color, position) is used to show what matters most.

Also commonly asked: **12 principles of visual design** (balance, contrast, emphasis, movement, white space, proportion, hierarchy, repetition, rhythm, pattern, unity, variety) — useful for a diagram-based answer.

 **Study more:**

- [Principles of UI/UX Design — GeeksforGeeks](#)
- [12 Principles of Visual Design Every UI Designer Should Know — GeeksforGeeks](#)

Unit II: Introduction to Information Architecture

2.1 What is Information Architecture?

Information Architecture (IA) is the practice of organizing, structuring, and labeling content so that people can find information and complete tasks efficiently. Think of IA as the **blueprint of a building** — but instead of rooms and hallways, it defines categories, navigation paths, and content relationships within a website, app, or system.

IA answers three core questions for a user:

- *Where am I?*
- *What's here?*

- *Where can I go next?*

IA is closely tied to, but distinct from, UX design: **UX is the umbrella discipline** (how the product feels overall), while **IA is one of its foundational pillars** (how content is structured underneath).

Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#)
 - [What is Information Architecture in UX Design? — GeeksforGeeks](#)
 - [Difference between IA and UX Design — GeeksforGeeks](#)
-

2.2 Principles of Information Architecture

The classic **8 Principles of Information Architecture** (Dan Brown's IA principles, foundational for this subject) are commonly tested:

1. **Principle of Objects** — treat content as living, evolving objects with their own lifecycle, attributes, and behaviors.
2. **Principle of Choices** — keep menus/choices small and meaningful; too many options overwhelm the user.
3. **Principle of Disclosure** — show enough information to help users understand what kind of information they'll find if they dig deeper (progressive disclosure).
4. **Principle of Exemplars** — describe the contents of categories by giving examples of the content.
5. **Principle of Front Doors** — assume at least 50% of users will enter through a page other than the homepage, so every page must give context.
6. **Principle of Multiple Classification** — offer multiple classification schemes so users with different mental models can find the same content.
7. **Principle of Focused Navigation** — keep one navigation system focused on one type of classification, rather than mixing schemes.
8. **Principle of Growth** — design the structure so it can scale as more content is added.

Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#) (*covers IA fundamentals and organization principles*)
 - [Information Architecture — A Practical Guide](#) (*explains the 8 principles in more depth*)
-

2.3 Role of the Information Architect

An **Information Architect (IA)** is the person responsible for organizing content so it makes sense to users and supports business goals. Their responsibilities include:

- Conducting **user research** (interviews, surveys, card sorting) to understand mental models.
- Creating **sitemaps, taxonomies, and content inventories**.
- Defining **navigation, labeling, and search systems**.
- Working closely with **UX designers, content strategists, developers, and stakeholders**.
- Validating structures through **tree testing** and usability testing.

- Balancing **business requirements** (what the organization wants to promote) against **user needs** (what visitors are actually looking for).

An IA essentially acts as a translator between business goals, content, and user behavior.

Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#)
- [A Beginner's Guide to Information Architecture in UX — Loop11](#)

2.4 Areas of Information Architecture

IA is typically applied across these areas:

1. **Content organization** — grouping and categorizing content logically.
2. **Navigation design** — menus, links, breadcrumbs that let users move around.
3. **Labeling** — the words/icons used to represent categories and links.
4. **Search systems** — enabling users to find content directly via queries.
5. **Metadata & tagging** — descriptive data attached to content for filtering/search.
6. **Taxonomy/vocabulary control** — maintaining consistent terms across the site.

These areas map onto the "**four systems of IA**" that Rosenfeld & Morville (the standard IA textbook authors) define: **Organization Systems, Labeling Systems, Navigation Systems, and Search Systems** — this exact 4-way split recurs throughout Units III and IV of this syllabus.

Study more:

- [What is Information Architecture in UX Design? — GeeksforGeeks](#)
- [What is Information Architecture? A Beginner's Guide — Big Sea](#)

2.5 Types of Architecture: System, Enterprise, Application, Internet

This is a frequently confused, high-weightage topic. Learn the **scope difference**:

Type	Scope	Focus
System Architecture	A single system or application	Structure, components, modules, interfaces, and data flow of <i>one</i> system; ensures it meets performance and integration needs.
Enterprise Architecture (EA)	The entire organization	Aligns business strategy, processes, data, applications, and technology infrastructure across <i>all</i> systems in the company. Provides a top-down, holistic blueprint.
Application Architecture	A single application's internal design	Defines how an individual application's modules, layers (presentation/business/data), and components interact internally.
Internet Architecture	Networked, distributed systems	Defines how data, protocols (TCP/IP, HTTP), servers, and clients communicate to deliver content over the internet.

Type	Scope	Focus
	on the web	

Key relationship to remember: Enterprise Architecture is the broadest (covers the whole company across all systems); System/Application Architecture zooms into individual systems; Internet Architecture concerns the network/communication layer that connects everything.

Study more:

- [Difference between System Architecture and Software Architecture — GeeksforGeeks](#)
- [Enterprise Architecture vs. Distributed System — GeeksforGeeks](#)
- [Enterprise Architect Framework — GeeksforGeeks](#)
- [Enterprise Architecture, Solution Architecture, System Architecture — Differences \(Digital Bank Expert\)](#)

2.6 Research and Practice in Information Architecture

IA is both an **academic research discipline** and a **hands-on practice**:

Research side studies:

- How users form mental models of information spaces.
- Effectiveness of different classification/navigation schemes (evaluated through controlled studies).
- Card sorting and tree-testing methodologies to validate structures scientifically.

Practice side involves:

- Real-world deliverables: sitemaps, wireframes, content inventories, taxonomy documents.
- Iterative testing with real users (usability testing, A/B testing of navigation).
- Continuous evolution of IA as content grows (governance).

Card Sorting (a key practical research technique) comes in two types:

- **Open card sorting** — users create their own categories and group items into them (used to *discover* a structure).
- **Closed card sorting** — users sort items into categories that are already defined (used to *validate* a structure).

Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#) (*has a dedicated card-sorting section*)
- [Mastering Card Sorting Methods for Effective UX Design — CleverX](#)

Unit III: Organizing Information

3.1 Organizing Information & Organizational Challenges

Organizing information means grouping content into logical, predictable categories. The **main organization schemes** are:

- **Exact organization schemes** (unambiguous — users know exactly where to look):
 - *Alphabetical* (e.g., a phone directory)
 - *Chronological* (e.g., a news archive by date)
 - *Geographical* (e.g., store locator by city)
- **Ambiguous organization schemes** (require editorial judgment — harder to design but often more useful):
 - *Topical* (organize by subject, e.g., "Sports", "Politics")
 - *Task-oriented* (organize by what the user wants to do, e.g., "Track My Order")
 - *Audience-specific* (organize by who the visitor is, e.g., "For Students" / "For Faculty")
 - *Metaphor-driven* (using a familiar real-world concept as the organizing idea)

Organizational challenges commonly arise because:

1. **Ambiguity of language** — the same word means different things to different users (heterogeneity).
2. **Differences in perspective** — a marketing team might want product-first organization; a support team wants issue-first organization.
3. **Internal politics** — departments want their content to be prominent, leading to biased structures.
4. **Constant content growth** — schemes that work for 50 pages break down at 5,000 pages.

Study more:

- [What is Information Architecture? A Beginner's Guide — Big Sea](#) (*explains organizational systems clearly*)
 - [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#)
-

3.2 Organizing Websites and Intranets

Websites are usually organized for a broad, external, less predictable audience — so simpler, more intuitive schemes (topical, audience-based) work best, and search plays a big role because visitors may land on any page via search engines.

Intranets are organized for a known, internal audience (employees) — so organization can be more specific to internal processes, departments, and workflows (e.g., organized by department, project, or HR process), since the user base and their vocabulary are well understood.

Key design decision for both: choosing between a **top-down approach** (start from business goals and structure content into categories) and a **bottom-up approach** (start with the actual content/pages you have and let the structure emerge). Most real projects use a mix of both.

Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#)
 - [A Beginner's Guide to Information Architecture in UX — Loop11](#)
-

3.3 Creating Cohesive Organization Systems

A **cohesive organization system** doesn't rely on just one scheme — it layers multiple schemes together so users with different needs and mental models can all succeed. Best practices:

- Combine an **exact scheme** (e.g., A–Z index) with an **ambiguous scheme** (e.g., topical categories) so users have multiple paths to the same content.
- Keep the **hierarchy balanced** — not too shallow (everything dumped on one level) and not too deep (content buried 6 clicks deep).
- Maintain **consistent granularity** — categories at the same level should represent roughly the same scope of content.
- Use **cross-listing** (placing the same content under more than one category) when content logically belongs in multiple places.
- Validate structure using **card sorting** (to design) and **tree testing** (to verify users can find things).



Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#)
 - [A Beginner's Guide To Information Architecture in UX — Loop11](#)
-

3.4 Organizing the WWW & Browser Navigation Features

Organizing the **World Wide Web** at large is far messier than organizing a single site — there is no central authority, content is added constantly, and different sites use different schemes. This is why **search engines** (Google, Bing) and **directories** became essential — they impose a searchable layer over an inherently unorganized web.

Browser navigation features that support a user moving through this unorganized space:

- **Back/Forward buttons** — move through browsing history.
- **Bookmarks/Favorites** — let users save and revisit important pages directly, bypassing site navigation.
- **Address bar & autocomplete** — lets users jump directly to a known URL.
- **Tabs** — allow parallel browsing of multiple pages/sites.
- **History** — a chronological organization scheme applied automatically by the browser.

These browser-level tools exist precisely *because* the web itself lacks a single, consistent IA — users need their own personal navigation aids.



Study more:

- [Navigation Element in Web Design — GeeksforGeeks](#)
 - [Search Engine — GeeksforGeeks](#)
-

3.5 Building Context & Improving Flexibility

Building context means ensuring that no matter which page a user lands on, they always know:

- Where they are within the larger structure.
- What organization/site they're browsing (branding, header).
- How to get back to a higher level.

Techniques to build context:

- Consistent **headers, footers, and logos** across every page.
- **Breadcrumb trails** showing the path from the homepage to the current page.
- **"You are here" highlighting** in navigation menus.
- **Page titles and headings** that clearly describe the current content.

Improving flexibility means giving users multiple ways to reach the same content, since different users have different needs at different times:

- Supplementary navigation (related links, "see also" sections).
- Multiple classification schemes (as discussed in 2.2 — Principle of Multiple Classification).
- Search as a fallback when navigation doesn't work for a user.

Study more:

- [Breadcrumb Navigation in Design — GeeksforGeeks](#)
 - [How to Create Intuitive Navigation? — GeeksforGeeks](#)
-

3.6 Types of Navigation Systems

Navigation systems are traditionally classified into four types (from Rosenfeld & Morville's IA framework, which this course closely follows):

1. **Hierarchical (Structural) Navigation** — the primary navigation derived directly from the site's information hierarchy (parent → child categories). This is the "main" navigation most sites rely on.
2. **Global Navigation** — a site-wide system (usually in the header) present on every page, giving access to top-level sections regardless of where the user currently is.
3. **Local Navigation** — navigation specific to a sub-section of the site, letting users move within a related group of pages (e.g., a sidebar menu within a documentation section).
4. **Contextual (Ad Hoc / Embedded) Navigation** — links embedded within content itself, connecting to specific, related pages that the hierarchy alone wouldn't surface (e.g., "related articles" or inline hyperlinks).

Additional commonly used types in modern web design: horizontal navbars, dropdown menus, hamburger menus, vertical sidebars, breadcrumb navigation, tab navigation, and footer navigation.

Study more:

- [Navigation Element in Web Design — GeeksforGeeks](#)
 - [Types of Navigation Systems — *Information Architecture for the World Wide Web* \(O'Reilly, the standard IA textbook\)](#)
-

3.7 Integrated & Remote Navigation Elements

- **Integrated Navigation Elements** are built directly into the page layout as part of the main design — e.g., the global navbar, local sidebar menus, and breadcrumbs that appear consistently within the page template itself.

- **Remote Navigation Elements** exist *outside* the integrated flow — supplementary tools placed separately from the main page layout to give users an alternate way to navigate:
 - **Sitemaps** — a page listing the entire site hierarchy, useful for both users and search engines.
 - **Indexes** — an alphabetical listing of topics/keywords with links (like a book's index).
 - **Guided navigation/tours** — structured, sequential paths through content, often used for tutorials or onboarding.

Remote elements are especially useful as a *safety net* for users who get lost using integrated navigation alone — they provide a "big picture" or alternative access route.

Study more:

- [Navigation Element in Web Design — GeeksforGeeks](#)
- [Types of Navigation Systems — O'Reilly IA book](#)

3.8 Designing Elegant Navigation Systems

An "elegant" navigation system balances **simplicity** with **completeness**. Guidelines:

1. **Keep the hierarchy visible** — don't let global/local/contextual navigation contradict the underlying content hierarchy.
2. **Be consistent** — same placement, styling, and behavior of navigation elements across the whole site.
3. **Limit choices per screen** — Miller's "7±2" rule: humans handle roughly 5–9 options at once comfortably.
4. **Provide feedback** — highlight the current/active section so users always know "where am I."
5. **Design for scanning, not reading** — users scan menus quickly; use clear, front-loaded labels.
6. **Support multiple navigation modes together** — hierarchical + global + local + contextual + search, working in harmony rather than competing for attention.
7. **Test with real users** — via tree testing, to confirm the navigation actually helps people complete tasks.

Study more:

- [How to Create Intuitive Navigation? — GeeksforGeeks](#)
- [Navigation Element in Web Design — GeeksforGeeks](#)

Unit IV: Labeling Systems & Search

4.1 Labeling Systems — "Not Labels"

A **labeling system** is the *set of words, phrases, and icons* used to represent chunks of content — such as navigation menu items, headings, and links. The syllabus explicitly distinguishes "**labeling systems**, not labels" because a labeling system is not just one label; it is the **overall, consistent vocabulary** applied across an entire site.

Why labels matter so much: labels are usually the **only representation of content** a user sees before clicking — a poor label can make excellent content invisible or misleading. Good labeling reduces cognitive load and helps users predict what they'll find before they click.

 Study more:

- [Information Architecture: A Practical Guide \(labeling systems section\)](#)
 - [What is Information Architecture in UX Design? — GeeksforGeeks](#)
-

4.2 Types of Labeling Systems

Labels appear in websites in two formats — **textual** and **iconic** — and typically serve one of these roles:

1. **Labels as contextual links** — the anchor text of a hyperlink (e.g., "Read More," "Contact Us").
2. **Labels within navigation systems** — the menu/nav item names (e.g., "Products," "About Us").
3. **Labels as headings** — text that identifies and breaks up a block of content on a page.
4. **Labels as index terms** — keywords/metadata used behind the scenes for search and filtering (not always visible to the user).
5. **Iconic labels** — images/icons used instead of or alongside text to represent an action or category (e.g., a magnifying glass icon for "search," a gear icon for "settings").

 Study more:

- [Types of Labeling Systems — *Information Architecture for the World Wide Web* \(O'Reilly\)](#)
 - [Information Architecture: A Practical Guide](#)
-

4.3 Creating Effective Labeling Systems

Best practices for designing labels:

1. **Narrow the scope** — focus on a specific content domain and use consistent terminology within it, rather than borrowing inconsistent terms from many sources.
2. **Develop consistent style rules** — decide once, and apply everywhere:
 - Case (Title Case vs. sentence case)
 - Verb form (e.g., always "Ordering" vs. always "Order" for gerund-style labels)
 - Length limits (e.g., all nav labels under 2–3 words)
3. **Use familiar, user-tested terminology** — pull terms from actual user language (via card sorting or search-log analysis), not internal jargon.
4. **Sources for good labels:**
 - Labels already in use on the site (existing conventions).
 - Competitor/other websites in the same domain.
 - Controlled vocabularies and thesauri (industry standard term lists).
 - Content itself (extracting recurring terms from the actual text).
 - Users and subject-matter experts (via interviews and testing).

 Study more:

- [Blog: Information Architecture — labelling table exercise \(Medium\)](#)
 - [Types of Labeling Systems — O'Reilly IA book](#)
-

4.4 Fine-Tuning the Labeling System

Once a labeling system is drafted, it must be refined iteratively:

- **Test labels with real users** — via card sorting (do users group items the way the labels suggest?) and first-click testing (do users click the correct label to find what they need?).
- **Check for consistency** — audit every label across the site for style, tone, and terminology drift.
- **Watch for ambiguity** — a label that makes sense internally may confuse external users (e.g., "Solutions" vs. "Products").
- **Iterate based on analytics** — track which labels get low click-through or high bounce rates, and revise them.
- **Maintain a living style guide/glossary** of approved labels so future content stays consistent as the site grows.

Study more:

- [Information Architecture: A Complete Guide For Beginners — GeeksforGeeks](#) (*card sorting/testing sections apply directly*)
 - [Information Architecture: A Practical Guide](#)
-

4.5 Non-Representational Labeling Systems & The Double Challenge

Non-representational labeling systems don't describe content directly — instead they use arbitrary codes, numbers, or internal names that carry no inherent meaning to the outside user (e.g., product codes like "SKU-4471," or internal project codenames). These are sometimes unavoidable (e.g., legal/technical document numbering) but should be minimized in user-facing navigation because they add no descriptive value.

The "Double Challenge" in labeling refers to the fact that a good information architect must solve **two problems simultaneously**:

1. Design a labeling system that fits the **content and structure** correctly (an internal, organizational challenge).
2. Ensure the labels make sense to and are usable by the **actual end users**, who may have very different mental models than the people who built the system (an external, user-facing challenge).

Balancing internal organizational logic against outside user comprehension is what makes labeling genuinely difficult — a system can be perfectly logical to its creators and still fail with real users.

Study more:

- [Information Architecture: A Practical Guide](#)
 - [Types of Labeling Systems — O'Reilly IA book](#)
-

4.6 Searching a Website — Understanding How Users Search

Before designing search, you must understand **why and how users search**. Users typically fall into these search-behavior patterns:

- **Known-item searching** — the user knows exactly what they want (e.g., a specific document title).
- **Existence searching** — the user isn't sure if the information exists at all, and is searching to find out.

- **Exploratory searching** — the user has a general topic in mind but isn't sure exactly what they're looking for; they refine as they browse results.
- **Comprehensive (research) searching** — the user wants to find *everything* related to a topic, not just one answer.

Because these needs differ so much, **search and browse (navigation) are deeply integrated** — many users bounce between searching and browsing multiple times before finding what they want (an iterative process, not a single query-response event).

Study more:

- [What are Search Engines? — GeeksforGeeks](#)
 - [Understanding Search Engines — GeeksforGeeks](#)
-

4.7 Designing the Search Interface

Guidelines for a good search interface (frequently asked as a design/critique question in exams):

1. **Make the search box visible and easy to find** — typically top-right or top-center of the page.
2. **Set proper user expectations** — indicate what will be searched (the whole site, or just a section) via placeholder text or scoped search options.
3. **Support autocomplete/suggestions** — helps users refine queries and reduces errors.
4. **Display results clearly** — show relevance-ranked results with useful snippets, not just raw titles.
5. **Allow filtering/faceted search** — let users narrow results by category, date, price, etc.
6. **Handle zero-result queries gracefully** — suggest spelling corrections or related content instead of a blank "no results" page.
7. **Provide feedback on the query itself** — echo back what was searched, and show the number of results found.
8. **Keep the search UI consistent** with the site's overall look and feel.

Technically, a search engine/interface is composed of three components: **Web Crawler** (gathers content), **Database/Index** (stores it), and **Search Interface** (lets users query it) — the same architecture applies whether it's Google or a site's internal search box.

Study more:

- [Components of Search Engine — GeeksforGeeks](#)
 - [What are Search Engines? — GeeksforGeeks](#)
-

4.8 The Reference Interview & Indexing the Right Stuff

The Reference Interview is a concept borrowed from library science: rather than assuming you know what the user *really* wants from their initial query, a good search system (or search designer) tries to clarify intent — through query refinement suggestions, filters, and clarifying prompts — similar to how a librarian asks follow-up questions to understand a patron's actual need before pointing them to a resource.

Indexing the right stuff means being deliberate about *what content actually gets indexed* for search:

- Not all content should be searchable (e.g., navigation boilerplate, footers, duplicate pages should usually be excluded).
- Metadata (titles, tags, descriptions) should be rich enough to support accurate matching.
- Index freshness matters — outdated indexes return outdated/broken results.
- Different content types (PDFs, images, videos) need different indexing strategies (e.g., OCR or alt-text for non-text content).

**Study more:**

- [Search Engine — GeeksforGeeks](#)
- [Designing Distributed Search System — GeeksforGeeks](#)

4.9 To Search or Not to Search

A critical, often-overlooked IA decision: **search is not always the right solution**. Considerations before adding a search feature:

Arguments for including search:

- Large, frequently changing content (too big to browse manually).
- Users often know precisely what they're looking for (known-item searching).
- Site has enough content volume to make indexing worthwhile.

Arguments against (or for delaying) search:

- Small sites — a good navigation/hierarchy alone may suffice, and a poorly implemented search can actually hurt usability more than no search at all.
- Poor-quality/inconsistent content — "garbage in, garbage out"; search over badly organized or duplicate content produces frustrating results.
- Not enough resources to *maintain* the search index and continuously tune relevance.

Rule of thumb: A bad search experience is often worse than no search at all, because it creates a false expectation of easy retrieval that then fails. Site owners should only invest in search once navigation/labeling are solid and there's enough content volume/change frequency to justify it.

**Study more:**

- [What are Search Engines? — GeeksforGeeks](#)
- [Components of Search Engine — GeeksforGeeks](#)

Unit V: The Open Group Architecture Framework (TOGAF)

5.1 Introduction to TOGAF 9

TOGAF (The Open Group Architecture Framework) is the world's most widely adopted **Enterprise Architecture (EA) framework**. It provides a structured approach for **designing, planning, implementing, and governing** an enterprise's information technology architecture, ensuring IT stays aligned with business goals.

Key facts:

- Developed and maintained by **The Open Group**, first released in 1995, based on the U.S. DoD's TAFIM framework.
- Models enterprise architecture at **four levels: Business, Data, Application, and Technology** (together often called "BDAT").
- Adopted by a majority of Fortune 500 and Global 50 companies.
- Free to use internally within an organization (commercial/resale use requires Open Group licensing).

**Study more:**

- [Enterprise Architect TOGAF \(The Open Group Architecture Framework\) — GeeksforGeeks](#)
- [TOGAF — Official page, The Open Group](#)

5.2 TOGAF 9 Management Overview

At a management level, TOGAF exists to solve a business problem: **without a common framework, every enterprise architecture project reinvents its own process, vocabulary, and deliverables**, making projects hard to compare, govern, or reuse across the organization.

TOGAF's management-level value proposition:

- **Standardization** — a common method and vocabulary for all architecture work in the enterprise.
- **Alignment** — ensures IT investment directly supports business strategy.
- **Risk reduction** — a proven, repeatable method reduces the risk of architecture projects failing or missing requirements.
- **Resource optimization** — avoids duplicated effort by reusing architecture assets (via the Enterprise Continuum, see 5.7).
- **Governance** — provides checkpoints to ensure architecture work stays compliant with principles and standards (see 5.8).

Management typically engages with TOGAF through its outputs — the **Statement of Architecture Work**, architecture roadmaps, and governance approvals — rather than the detailed technical artifacts.

**Study more:**

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
- [What is TOGAF? An EA framework for aligning technology to business — CIO.com](#)

5.3 TOGAF 9 Components

TOGAF is made up of several interlocking components:

1. **Architecture Development Method (ADM)** — the core, step-by-step, iterative method for developing an enterprise architecture (see 5.4–5.5).
2. **ADM Guidelines & Techniques** — supporting guidance for adapting the ADM to different situations (e.g., security architecture, agile projects).
3. **Architecture Content Framework** — defines the structure of architecture deliverables, artifacts, and building blocks produced during ADM.
4. **Enterprise Continuum** — a categorization scheme/repository that classifies architecture and solution assets from generic (foundational) to organization-specific (see 5.7).

5. **TOGAF Reference Models** — including the **Technical Reference Model (TRM)** and **Integrated Information Infrastructure Reference Model (III-RM)**, offering pre-built reference architectures.
6. **Architecture Capability Framework** — defines the organizational structures, roles, skills, and processes needed to actually operate and sustain the architecture practice.

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)

5.4 Introduction to the ADM

The **Architecture Development Method (ADM)** is the **heart of TOGAF** — a proven, iterative, cyclical process for developing and managing the entire lifecycle of an enterprise architecture. It is not a rigid waterfall process; organizations are expected to **tailor** the ADM to fit their own context (adding/removing steps, aligning with existing frameworks like Agile-Scrum, ITIL, or PMBOK).

The ADM cycle revolves around a central hub called the **Requirements Management** phase, which continuously feeds validated requirements into and out of every other phase — ensuring the architecture stays grounded in real business needs throughout.

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
- [The TOGAF Standard 9.2 — Official ADM documentation, The Open Group](#)

5.5 ADM Phases

The ADM cycle has **10 phases** arranged in a circle around the central Requirements Management phase:

Phase	Name	Purpose
Preliminary Phase	Framework & Principles	Prepares the organization: defines architecture principles, scope, and tailors the ADM to organizational needs before the cycle begins.
Phase A	Architecture Vision	Defines the scope, identifies stakeholders, creates a high-level vision of what the architecture will achieve, and secures approval to proceed.
Phase B	Business Architecture	Develops the target business architecture — describing business strategy, governance, processes, and organization.
Phase C	Information Systems Architecture	Develops target Data and Application architectures — what information is needed and what applications will manage it.
Phase D	Technology Architecture	Develops the target technology architecture — hardware, software, networks needed to support the applications and data.

Phase	Name	Purpose
Phase E	Opportunities & Solutions	Identifies major implementation projects and groups them into a coherent roadmap, evaluating build-vs-buy options.
Phase F	Migration Planning	Prioritizes projects, assesses costs/benefits/risks, and finalizes a detailed implementation and migration plan.
Phase G	Implementation Governance	Provides architectural oversight of the actual implementation, ensuring projects conform to the architecture.
Phase H	Architecture Change Management	Monitors the running architecture, manages change requests, and decides whether a new ADM cycle is required.
Requirements Management	(central, continuous)	Manages architecture requirements throughout — feeding into and pulling from every other phase.

Memory tip for exams: Preliminary → A (Vision) → B (Business) → C (Info Systems: Data + Apps) → D (Technology) → E (Opportunities) → F (Migration) → G (Governance) → H (Change Mgmt) → back to Requirements Management (center).

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
- [The TOGAF Standard 9.2 — Phase A: Architecture Vision \(Official, The Open Group\)](#)
- [10 Phases of TOGAF 9.2 ADM Explained — KnowledgeHut](#)

5.6 ADM Deliverables

Each ADM phase produces specific **deliverables** — formal, tangible work products that get reviewed, approved, and stored in the architecture repository. Common deliverables across the cycle include:

- **Statement of Architecture Work** (Phase A) — the formal agreement to begin the architecture project.
- **Architecture Vision document** (Phase A) — the high-level, aspirational description of the target state.
- **Business, Data, Application, and Technology Architecture documents** (Phases B, C, D) — the detailed target-state descriptions for each domain.
- **Architecture Requirements Specification** — the consolidated, validated requirements (from Requirements Management).
- **Migration Plan / Implementation Roadmap** (Phases E, F) — the sequenced plan for moving from baseline to target architecture.
- **Architecture Contract** (Phase G) — a formal agreement between architecture and implementation teams ensuring delivered systems conform to the architecture.
- **Architecture Definition Document** — a comprehensive, consolidated view of the overall target architecture, gathered across phases.

Deliverables differ from **artifacts** (smaller, granular architecture work products like catalogs, matrices, and diagrams) and **building blocks** (reusable components, see 5.9) — a deliverable is typically a *contractually agreed-upon output*, while artifacts/building blocks are the internal content that composes it.

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
 - [The TOGAF Standard 9.2 — Official documentation, The Open Group](#)
-

5.7 Enterprise Continuum & Architecture Repository

The **Enterprise Continuum** is a conceptual tool that classifies and organizes all architecture and solution assets on a **spectrum from generic to organization-specific**:

- **Foundation Architectures** (most generic — broadly applicable, e.g., TOGAF's own Technical Reference Model).
- **Common Systems Architectures** (generic but applicable to a specific domain, e.g., "common security architecture").
- **Industry Architectures** (specific to an industry vertical, e.g., banking or healthcare reference models).
- **Organization-Specific Architectures** (the most specific — tailored fully to one enterprise's unique needs).

The idea: rather than building every architecture from scratch, architects **reuse and specialize assets** as they move along this continuum, saving time and increasing consistency.

The **Architecture Repository** is the actual storage system/database that holds all these architecture outputs — reference models, standards, governance logs, past architecture deliverables, and the organization's own architecture landscape — organized according to the Enterprise Continuum's classification.

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
 - [TOGAF as an Enterprise Architecture Framework — Official, The Open Group](#)
-

5.8 Architecture Governance

Architecture Governance is the practice of ensuring that architecture work is executed consistently, complies with established principles/standards, and stays aligned with business objectives — throughout the entire ADM lifecycle, not just at the end.

Core governance activities:

- **Compliance reviews** — checking that new systems/projects conform to the approved architecture.
- **Managing architecture contracts** — enforcing agreements between architecture and delivery teams (Phase G).
- **Dispensation process** — a formal way to grant temporary exceptions when a project cannot fully comply with the architecture, with tracked justification.
- **Monitoring change** — via Phase H (Architecture Change Management), governance decides when changes are significant enough to trigger a new ADM cycle.

- **Maintaining an audit trail** of architecture decisions for accountability.

Good governance is what prevents an enterprise architecture from becoming "shelf-ware" — a nicely documented plan that nobody actually follows.

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
- [The TOGAF Standard — Official, The Open Group](#)

5.9 Views, Viewpoints & Architecture Building Blocks

- **Viewpoint** — a *specification* or template that defines how to construct and use a particular kind of view — essentially, "the perspective/lens" (e.g., a "security viewpoint" defines what a security-focused diagram should always show).
- **View** — the *actual representation* of the architecture created by applying a viewpoint to real, specific architecture content (e.g., the actual security diagram for a specific project, built using the security viewpoint's rules).

Analogy: a **viewpoint** is like a blueprint template for how to draw a floor plan; a **view** is the *actual floor plan drawing* for a specific building, created using that template. Different stakeholders (business users, developers, security teams, network engineers) need different views/viewpoints of the same underlying architecture to understand what matters to them.

Architecture Building Blocks (ABBs) are reusable, well-defined components of architecture — potentially reusable capabilities (e.g., "Customer Identity Management") — that are combined and refined into **Solution Building Blocks (SBBs)**, which represent actual, real-world components/products that implement the ABB in a specific solution.

Study more:

- [Enterprise Architect TOGAF — GeeksforGeeks](#)
- [The TOGAF Standard 9.2 — Official documentation, The Open Group](#)

Quick Revision Sheet

Unit	Core Idea to Remember
I	UI = look; UX = whole journey. Design principles: Simplicity, Consistency, Feedback, Accessibility, Hierarchy.
II	IA = organizing + labeling + navigation + search. 8 IA principles (Objects, Choices, Disclosure, Exemplars, Front Doors, Multiple Classification, Focused Navigation, Growth). System < Application < Enterprise Architecture in scope.
III	4 organization schemes (Exact: alphabetical/chronological/geographical; Ambiguous: topical/task/audience/metaphor). 4 navigation types (Hierarchical, Global, Local, Contextual). Remote elements = sitemap, index, guides.

Unit	Core Idea to Remember
IV	Labels = words/icons representing content. Types: contextual links, nav labels, headings, index terms, icons. Search behaviors: known-item, existence, exploratory, comprehensive. Search isn't always the right answer.
V	TOGAF = EA framework by The Open Group. ADM = 10 phases (Preliminary, A–H) around central Requirements Management. Enterprise Continuum = generic → specific asset classification. Viewpoint = template; View = actual diagram.

Compiled for exam preparation — cross-check university-specific lecture slides for any local terminology variations, but the concepts above align with the standard Information Architecture textbook (Rosenfeld & Morville) and the official TOGAF 9.2 standard.